TD/TP2. Modéliser la ligne d'embouteillage dans Azure Digital Twins

Digital Twins, séance 2. Durée : 2 heures, en binôme.

En séance 1, vos capteurs simulés publiaient leurs mesures sur MQTT. Mais rien ne disait à quelle machine appartenait pression-01, ni que la remplisseuse alimente la capsuleuse. Aujourd'hui, vous donnez une structure à la ligne.

La séance a deux parties. Dans le TD, vous modélisez la ligne : sur papier d'abord, puis en DTDL. Dans le TP, vous construisez le graphe dans votre propre instance Azure Digital Twins, et vous l'interrogez.

Ce graphe servira en séance 3 : c'est lui que vos mesures MQTT mettront à jour. Respectez donc exactement les noms et les identifiants donnés dans cet énoncé.

Ce que vous saurez faire à la fin

- classer une donnée de la ligne : propriété, relation, composant, ou rien dans le jumeau ;
- écrire et vérifier un modèle DTDL v3, avec de l'héritage ;
- créer une instance Azure Digital Twins et vous donner le bon rôle ;
- construire un graphe de jumeaux en ligne de commande, puis l'interroger.

Avant de commencer

Il vous faut votre abonnement Azure for Students activé, Azure CLI connectée (az login) et VS Code. Travaillez dans un dossier td2-ligne, qui contiendra un sous-dossier modeles.

Utilisez un terminal Bash : Azure Cloud Shell dans le portail, WSL, Git Bash, ou le terminal de macOS ou de Linux.

Attention. Sous PowerShell ou dans l'invite de commandes de Windows, les guillemets du JSON sont interprétés autrement, et les commandes de cet énoncé échouent. Si vous devez rester sous PowerShell, mettez le JSON dans un fichier et passez-le avec '@fichier.json'.

Partie 1. TD : modéliser la ligne

Étape 1. Modéliser la ligne, puis écrire les modèles (30 min)

Exercice 1. Ranger les données (5 min)

Pour chaque donnée, dites ce qu'elle devient dans nos modèles : une propriété, une relation, un composant, ou rien dans le jumeau. Justifiez en une phrase.

Donnée	Votre réponse
a. le numéro de série de la remplisseuse	
b. la pression, mesurée toutes les 2 secondes	
c. le fait que la ligne contient la capsuleuse	
d. le seuil d'alerte des vibrations	
e. l'ordre « démarrer le remplissage »	
f. l'état de la capsuleuse : en marche, à l'arrêt ou en panne	
g. la tête de vissage de la capsuleuse	
h. l'historique des vibrations de la semaine	

Indice : souvenez-vous de ce qu'Azure Digital Twins fait, et ne fait pas, de la télémétrie et des commandes.

Exercice 2. Composant ou relation ? (5 min)

Pour chaque élément, choisissez entre un composant et un jumeau séparé relié par une relation. Appuyez-vous sur les trois questions vues en cours. Plusieurs réponses peuvent se défendre : ce qui compte, c'est votre justification.

- le capteur de pression de la remplisseuse ;
- le moteur de la capsuleuse, qu'on ne suit jamais séparément ;
- la caméra qui contrôle le niveau de remplissage, et qu'on déplace parfois d'une ligne à l'autre ;
- le pupitre de commande intégré à la remplisseuse.

Exercice 3. Écrire les cinq modèles (20 min)

Écrivez un fichier JSON par modèle dans le dossier modeles. Tous commencent par "@context": "dtmi:dtdl:context;3". Respectez exactement les noms : le TP et la séance 3 en dépendent.

Fichier	DTMI	Hérite de	Contenu
Ligne.json	`dtmi:esilv:ligne:Ligne;1`	aucun	propriétés nom (string), emplacement (string), dateMiseEnService (date) relation contient, vers Machine
Machine.json	`dtmi:esilv:ligne:Machine;1`	aucun	propriétés numeroSerie (string), etat (Enum de string : en_marche, arret, panne) relation estEquipeeDe, vers Capteur
Remplisseuse.json	`dtmi:esilv:ligne:Remplisseuse;1`	Machine	propriété cadenceConsigne (integer) relation alimente, vers Capsuleuse
Capsuleuse.json	`dtmi:esilv:ligne:Capsuleuse;1`	Machine	propriété diametreBouchon (double)
Capteur.json	`dtmi:esilv:ligne:Capteur;1`	aucun	propriétés typeMesure (string), unite (string), seuilAlerte (double), derniereValeur (double), derniereMesure (dateTime), enAlerte (boolean)

Pour démarrer, voici le début de Remplisseuse.json :

```
{
  "@context": "dtmi:dtdl:context;3",
  "@id": "dtmi:esilv:ligne:Remplisseuse;1",
  "@type": "Interface",
  "displayName": "Remplisseuse",
  "extends": "dtmi:esilv:ligne:Machine;1",
  "contents": [
    { "@type": "Property", "name": "cadenceConsigne", "schema": "integer" },
    ...
  ]
}
```

Pour l'Enum de l'état, reprenez la syntaxe vue en cours, sur la diapo « Les types de données vont du simple au composé ».

Étape 2. Vérifier les modèles (10 min)

Exercice 4. Trouver les erreurs

Un collègue a écrit le modèle d'une étiqueteuse, la machine qui pose les étiquettes après la capsuleuse. Azure Digital Twins le refuse. Le fichier contient sept erreurs : trouvez-les, puis corrigez-les.

```
{
  "@context": "dtmi:dtdl:context;3",
  "@id": "dtmi:esilv:ligne:Étiqueteuse;1",
  "@type": "Interface",
  "extends": "dtmi:esilv:ligne:Machine;1",
  "contents": [
    { "@type": "Property", "name": "numeroSerie", "schema": "string" },
    { "@type": "Property", "name": "vitesseÉtiquetage", "schema": "int" },
    { "@type": "Property", "name": "type etiquette", "schema": "string" }
    { "@type": "Relationship", "name": "controle",
      "target": "dtmi:esilv:ligne:Capteurs;1" }
  ]
}
```

Passez ensuite vos cinq modèles à la même liste de vérification :

- le JSON est valide : VS Code souligne les erreurs, ou lancez python -m json.tool modeles/Machine.json ;
- chaque fichier commence par "@context": "dtmi:dtdl:context;3" ;
- chaque DTMI commence par dtmi:, ne contient ni accent, ni espace, ni tiret, et finit par ;1 ;
- chaque name commence par une lettre et ne contient que des lettres sans accent, des chiffres et des tirets bas ;
- chaque schema est un type connu : string, integer, double, boolean, date, dateTime, ou un Enum ;
- chaque extends et chaque target désigne un DTMI qui existe dans vos fichiers, avec la même version ;
- aucun modèle enfant ne redéclare un nom déjà présent dans son parent.

La vérification finale aura lieu à l'étape 4 : Azure Digital Twins refuse tout modèle invalide, avec un message qui indique l'erreur.

Question 1. Dans nos modèles, la pression n'est pas une Telemetry : c'est la propriété dernièreValeur du capteur. Pourquoi ? Où ira l'historique des mesures ?

Partie 2. TP : construire la ligne dans Azure Digital Twins

Étape 3. Créer l'instance et vous donner le rôle (15 min)

Choisissez un nom d'instance unique, en minuscules, par exemple dt-ligne suivi du nom de votre binôme. Dans les commandes qui suivent, remplacez <instance> par ce nom.

```
az group create --name rg-dt-ligne --location westeurope

az dt create --dt-name <instance> \
  --resource-group rg-dt-ligne --location westeurope

az dt role-assignment create --dt-name <instance> \
  --assignee "<votre adresse>" --role "Azure Digital Twins Data Owner"
```

Votre adresse est celle de votre compte Azure : la commande az account show --query user.name l'affiche. La première commande az dt peut vous proposer d'installer l'extension azure-iot : acceptez.

Attention. Sans le rôle Data Owner, vous recevrez une erreur d'accès à l'étape suivante, même si c'est vous qui avez créé l'instance. Le rôle peut mettre une ou deux minutes à s'appliquer.

Si az dt create échoue :

- message sur une région non autorisée : votre abonnement étudiant limite les régions. Choisissez-en une autre parmi celles que propose le portail ;
- message MissingSubscriptionRegistration : lancez az provider register --namespace Microsoft.DigitalTwins, attendez une minute, puis recommencez.

Étape 4. Importer les modèles, créer les jumeaux et les relations (30 min)

Écrivez vos commandes dans un script construire_ligne.sh plutôt que de les taper une à une. Vous pourrez reconstruire le graphe d'un coup, et ce script est un livrable.

Importer les modèles

```
az dt model create -n <instance> --from-directory ./modeles
az dt model list -n <instance> --query "[].id"
```

Importez les cinq modèles en une seule commande : Azure Digital Twins résout lui-même les dépendances entre eux. S'il refuse un modèle, lisez le message, corrigez le fichier et relancez. Si la commande indique qu'un modèle existe déjà, supprimez les modèles avec az dt model delete-all -n <instance> avant de relancer.

Créer les huit jumeaux

Créez les jumeaux du tableau. Voici la commande pour le premier capteur :

```
az dt twin create -n <instance> --dtmi "dtmi:esilv:ligne:Capteur;1" \
  --twin-id pression-01 --properties \
  '{"typeMesure":"pression","unite":"bar","seuilAlerte":3.4,"enAlerte":false}'
```

Identifiant	Modèle	Propriétés de départ
`ligne1`	Ligne	nom : Ligne L1 ; emplacement : ESILV, usine 1 ; dateMiseEnService : 2024-09-02
`remplisseuse-01`	Remplisseuse	numeroSerie : RMP-2024-001 ; etat : en_marche ; cadenceConsigne : 3300
`capsuleuse-01`	Capsuleuse	numeroSerie : CPS-2024-001 ; etat : en_marche ; diametreBouchon : 28.0
`pression-01`	Capteur	typeMesure : pression ; unite : bar ; seuilAlerte : 3.4 ; enAlerte : false
`temperature-01`	Capteur	typeMesure : temperature ; unite : °C ; enAlerte : false
`cadence-01`	Capteur	typeMesure : cadence ; unite : bouteilles/h ; enAlerte : false
`vibrations-01`	Capteur	typeMesure : vibrations ; unite : mm/s ; seuilAlerte : 4.5 ; enAlerte : false
`couple-01`	Capteur	typeMesure : couple ; unite : N·m ; enAlerte : false

Les identifiants des capteurs sont ceux des messages MQTT du TD1, et typeMesure reprend la fin de leur topic. C'est ce qui permettra, en séance 3, de relier chaque message à son jumeau. Tous les capteurs n'ont pas de seuil d'alerte : une propriété peut rester sans valeur.

Attention. N'utilisez pas d'apostrophe dans le JSON des propriétés : en Bash, elle fermerait la chaîne.

Créer les huit relations

Voici la commande pour la première relation. Donnez à chaque relation un identifiant lisible, de la forme source-relation-cible.

```
az dt twin relationship create -n <instance> \
  --twin-id ligne1 --relationship contient --target remplisseuse-01 \
  --relationship-id ligne1-contient-remplisseuse-01
```

Source	Relation	Cible
`ligne1`	`contient`	`remplisseuse-01`
`ligne1`	`contient`	`capsuleuse-01`
`remplisseuse-01`	`alimente`	`capsuleuse-01`
`remplisseuse-01`	`estEquipeeDe`	`pression-01`
`remplisseuse-01`	`estEquipeeDe`	`temperature-01`
`remplisseuse-01`	`estEquipeeDe`	`cadence-01`
`capsuleuse-01`	`estEquipeeDe`	`vibrations-01`
`capsuleuse-01`	`estEquipeeDe`	`couple-01`

Depuis la page de votre instance dans le portail Azure, ouvrez Azure Digital Twins Explorer et lancez la requête par défaut. Vérifiez le graphe : huit jumeaux, huit relations. Faites-en une capture, c'est un livrable. ADT Explorer ne dessine pas le graphe des modèles DTDL v3, seulement celui des jumeaux : c'est normal.

Question 2. Pourquoi faut-il créer les jumeaux avant les relations ? Que se passe-t-il si vous créez une relation vers un jumeau qui n'existe pas encore ?

Question 3. La relation estEquipeeDe est déclarée dans le modèle Machine. Pourquoi pouvez-vous l'utiliser depuis remplisseuse-01 ? Pourriez-vous créer une relation contient de remplisseuse-01 vers pression-01 ?

Étape 5. Interroger le graphe (20 min)

Écrivez les requêtes qui répondent aux six questions suivantes. Lancez-les avec la commande ci-dessous, et notez chaque requête et son résultat dans un fichier requetes.md.

```
az dt twin query -n <instance> --query-command "SELECT ..."
```

	Question
R1	Combien y a-t-il de jumeaux dans l'instance ?
R2	Quels sont les capteurs ?
R3	Quelles sont les machines ? Recommencez en ajoutant exact dans IS_OF_MODEL.
R4	Quels capteurs équipent la capsuleuse ?
R5	Pour ligne1, quelles machines contient-elle, et quels capteurs équipent chacune ?
R6	Combien y a-t-il de relations ? Indice : FROM RELATIONSHIPS.

Quatre règles à garder en tête :

- une requête avec JOIN doit filtrer sur le $dtId d'au moins un jumeau ;
- SELECT * est interdit avec JOIN : nommez les collections à renvoyer, par exemple SELECT M, C ;
- entourez la requête de guillemets doubles, et les valeurs d'apostrophes droites ;
- en Bash, écrivez \$dtId : sans la barre oblique inverse, Bash remplace $dtId par une variable vide et la requête échoue.

Question 4. Pourquoi R3 renvoie-t-elle deux jumeaux sans exact, et aucun avec exact ?

Étape 6. Mettre à jour un capteur à la main et observer (15 min)

En séance 3, une fonction Azure mettra à jour les capteurs à chaque message MQTT. Pour l'instant, vous le faites à la main. La pression vient de monter à 3,52 bar :

```
az dt twin update -n <instance> --twin-id pression-01 --json-patch '[
  {"op":"add","path":"/derniereValeur","value":3.52},
  {"op":"add","path":"/derniereMesure","value":"2026-10-12T09:14:02Z"},
  {"op":"replace","path":"/enAlerte","value":true}]'
```

1. Affichez le jumeau avec az dt twin show -n <instance> --twin-id pression-01. Repérez les nouvelles valeurs, puis la partie $metadata.
2. Écrivez une requête qui renvoie les capteurs en alerte. Écrivez-en une seconde, qui part de ligne1 et renvoie chaque machine avec ses capteurs en alerte.
3. Passez la remplisseuse en panne : modifiez sa propriété etat.
4. Sur temperature-01, essayez replace au lieu de add pour dernieReValeur. Notez ce qui se passe.

Question 5. Pour changer le seuil d'alerte de la pression, faut-il modifier le modèle ou le jumeau ? Et pour ajouter une propriété fabricant à tous les capteurs ?

Question 6. Pourquoi utilise-t-on add et non replace pour derniereValeur ? Pourquoi replace fonctionne-t-il pour enAlerte ?

Question 7. Juste après votre mise à jour, une requête peut encore renvoyer l'ancienne valeur. Pourquoi ? Que faire ?

Fin de séance

Ne supprimez ni l'instance ni le groupe de ressources : ils servent en séance 3. Le service se paie à l'usage, donc une instance qu'on n'interroge pas ne coûte presque rien.

Attention. N'utilisez pas az group delete : cette commande supprime l'instance et tout ce qu'elle contient.

Pour aller plus loin

Si vous avez fini en avance, voici trois pistes, comptées en bonus :

- ajoutez l'étiqueteuse corrigée de l'exercice 4 : importez son modèle avec az dt model create -n <instance> --models Etiqueteuse.json, créez le jumeau etiqueteuse-01 et reliez-le à ligne1 par contient. R3 doit maintenant renvoyer trois machines ;
- écrivez une requête qui compte les capteurs, avec SELECT COUNT() et IS_OF_MODEL ;
- rendez votre script rejouable : au début, supprimez tous les jumeaux (az dt twin delete-all --yes), puis tous les modèles (az dt model delete-all --yes).

Ce que vous rendez

- le dossier modeles, avec vos cinq fichiers JSON ;
- construire_ligne.sh, avec toutes vos commandes, de la création de l'instance aux relations ;
- requetes.md, avec chaque requête des étapes 5 et 6 et son résultat ;
- une capture du graphe des jumeaux dans ADT Explorer ;
- un README.md, avec vos réponses aux exercices 1, 2 et 4 et aux questions 1 à 7.

Barème

Critère	Points
Exercices 1, 2 et 4 du TD	3
Cinq modèles conformes à la spécification, importés sans erreur	5
Instance, rôle, et graphe complet : huit jumeaux, huit relations	4
Requêtes R1 à R6 et requêtes de l'étape 6, avec leurs résultats	4
Mise à jour du capteur et de la machine (étape 6)	2
README : réponses aux questions 1 à 7	2
Total	**20**

Bonus : jusqu'à 2 points supplémentaires, dans la limite de 20.